HTTP Security Header Analysis

Analytical Breakdown: How CSP Helps Prevent Cross-Site Script Loading:

Objective:

The objective of this analysis is to explain how the Content-Security-Policy (CSP) HTTP response header helps reduce the risk of cross-site script loading and other forms of untrusted resource injection in web applications.

Content-Security-Policy Overview:

HTTP response headers allow a web server to provide security instructions to the client browser. Among the important security headers are Strict-Transport-Security, Content-Security-Policy, X-Frame-Options and X-Content-Type-Options. CSP is specifically used to restrict untrusted resource injection by controlling with resources a browser is permitted to load.

When a browser receives a CSP header, it applies the defined policy before loading resources used by the webpage. This provides an additional layer of protection between untrusted content and the browser runtime.

CSP and Cross-Site Scripting:

Cross-Site Scripting (XSS) occurs when untrusted user input is inserted into dynamic web content without proper handling, allowing active client-side code to reach the browser runtime. The provided material describes XSS as a situation where untrusted parameters populate web templates without sufficient conversation or sanitization.

CSP helps reduce this risk by limiting the sources from which scripts and other resources may be loaded. If a webpage attempts to load a script from a source that is not allowed by the active policy, the browser can prevent that resource from being executed.

For example, a restrictive policy can allow scripts only from approved application resources while rejecting scripts loaded from unexpected external locations. This helps reduce the impact of injected content attempting to load unauthorized JavaScript.

Security Value of CSP:

CSP strengthens browser-side protection by reducing the number of locations from which executable content can be loaded. It is particularly useful as part of a broader defense against untrusted resource injection and XSS-related activity.

However, CSP should not replace secure handling of user input. Input still be validated and safely encoded before being inserted into web content. The XSS sanitization

section also demonstrates that converting raw structural tokens into neutral equivalents prevents them from being interpreted as active browser content.

Test Result Interpretation:

The python script checks whether the target server provides the CSP header along with other important security headers. If the header is missing, the script identifies it as a missing security control. If it is configured, the script displays the beginning of the policy value returned by the server.

A missing CSP header means that this browser-side restriction is not being provided by the tested server. This does not by itself prove that the application is vulnerable to XSS, but it indicates that an additional defensive control is absent.

Conclusion:

Content-Security-Policy is an important HTTP security mechanism that helps limit untrusted script and resource loading in the browser. By defining which sources are permitted, CSP can reduce the impact of cross-site scripting and resource injection attempts. When combined with secure input handling and proper sanitization, CSP contributes to a stronger web application security posture.

Local Network Port & Service Scanning:

Objective:

The objective of this task was to identify open local network ports and evaluate service exposure within a controlled development environment. The scan was performed against the local loopback address 127.0.0.1, using a predefined list of ports.

Tools Used: Python, socket library, Local Python HTTP server

Method:

A python script was created using socket module. The script attempted TCP connections to the following ports: 22, 80, 443, 5432, 8080.

For each port, socket.connect_ex() was used to determine whether a connection could be established. A return value of 0 indicated that the port was open.

The target used for the scan was: "127.0.0.1" , which represents the local machine and kept the activity inside the local lab environment.

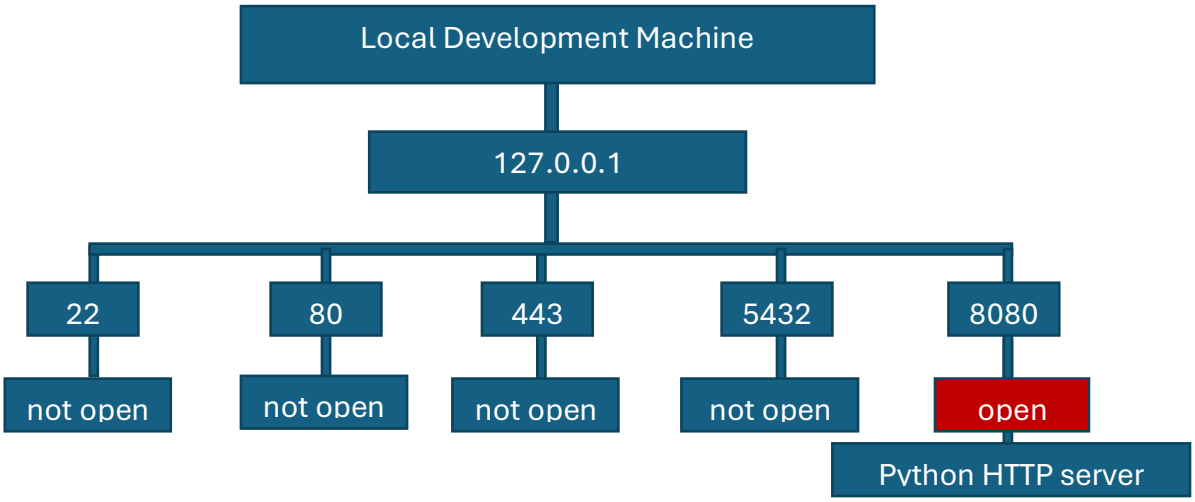
To create a known open service for testing, a local Python HTTP server was started on port 8080.

Result:

```
[*] Initiating Socket Sweep on: 127.0.0.1
[!] OPEN SERVICE DETECTED: Port 8080
```

Port 8080 was successfully detected as open because the local HTTP server was running on that port. Ports 22, 80, 443, and 5432 were not detected as open during the scan.

Network Topology:



The topology shows that the local machine was tested through its loopback interface and that port 8080 was the only tested port accepting TCP connections at the time of the scan.

Security Analysis:

Open ports represent services that are actively listening for connections. Identifying these services is useful for understanding the exposed network surface of a development environment.

An open port is not automatically vulnerable, but unnecessary or unintentionally exposed services can increase the attack surface. Regular port checks can help identify services that should be disabled, restricted, or reviewed.

Limitations:

The script only tested five predefined TCP ports. It did not scan the complete port range or identify detailed service versions. Also, ports that were not printed by the script should be described as not detected as open, rather than assuming everyone was permanently closed.

Conclusion:

The local port scan successfully detected the Python HTTP server running on port 8080. The exercise demonstrated how Python sockets can be used to identify active services and map network exposure within a controlled local environment. The network topology provides a simple representation of the tested ports and the service that was discovered.

SQL Injection (SQLi) Log Detection Engine:

Objective:

The objective of this task was to build a Python-based log detection engine capable of identifying suspicious patterns that may indicate SQL injection attempts. The script analyzes mock web access logs and searches for known SQLi indicators such as single quotes, SQL comments, OR conditions, and UNION SELECT statements.

Tools Used: Python, re module for regular-expression pattern matching

Method:

The script processes a list of simulated web access log entries. Each entry contains a source IP address, an HTTP request, and a response status code.

Before analyzing the logs, URL-encoded characters such as %20 and %27 are converted into spaces and single quotes. A regular expression is then used to search each decoded log entry for suspicious SQL-related patterns.

When a match is detected, the script extracts the source IP address and prints a critical alert together with the suspicious log entry.

The detector searches for patterns related to examples such as ' OR 1=1, UNION SELECT, and SQL comment characters.

Execution Result:

The first (normal) request was not identified as malicious. The other simulated requests contained SQL injection indicators and generated alerts. The detected entries included an authentication request containing an OR condition and a search request containing UNION SELECT.

This demonstrated that the detection engine could distinguish the normal sample request from the suspicious SQL-related patterns contained in the mock logs.

Unparameterized SQL Queries:

In an unparameterized SQL query, user-controlled input may be inserted directly into the SQL statement. If the input contains SQL operators or special characters, the database may interpret part of that input as SQL syntax rather than ordinary data.

For example, input containing a logical expression such as: ‘ OR ‘1’=’1 can change the intended structure of an SQL statement when user input and executable SQL are not properly separated.

This is the type of behavior the log detector attempts to recognize through suspicious patterns. The underlying risk comes from failing to separate execution logic from user-controlled parameters.

Prepared Statements:

Prepared statements separate the SQL command structure from the values supplied by the user. The SQL statement is defined first, while user input is supplied separately as parameter data.

This separation prevents normal parameter values from being interpreted as additional SQL instructions. As a result, characters such as quotes or SQL keywords contained in user input are handled as data rather than being allowed to modify the intended SQL structure.

The main execution difference is therefore:

Unparameterized query:
SQL structure + user input are combined

Prepared statement:
SQL structure and parameter values are separated

Prepared statements provide a stronger defensive approach because they reduce the possibility that user-controlled values can alter the SQL command being executed.

Limitations: The detection engine uses simple regular expression signatures. A match indicates suspicious content but does not prove that an actual database attack was successful.

The current rule set may also produce false positives because legitimate input can occasionally contain characters such as single quotes. A production detection system would require additional context and more advanced analysis.

Conclusion:

The SQLi log detection engine successfully identified suspicious patterns within simulated web access logs. The exercise demonstrated how defenders can inspect application logs for common SQL injection indicators and highlighted the security difference between directly combining user input with SQL statements and separating values through prepared statements.

Docker Container Misconfiguration:

Objective:

The objective of this task was to create a Python-based static analysis tool that inspects Dockerfile configuration directives and identifies potentially insecure container settings. The analysis focused on configuration choices that can increase deployment risk, including unpinned latest image tags, exposed SSH port 22, and the absence of an explicit USER instruction.

Tools Used: Python, Dockerfile

Method:

A Python script was created to read a Dockerfile line by line. Each line was cleaned and converted to uppercase so that configuration directives could be checked consistently. The scanner inspected the Dockerfile for three main conditions:

FROM ... :latest
EXPOSE 22
Absence of an explicit USER instruction

The script also kept track of line numbers so detected issues could be associated with their location in the Dockerfile. The analysis was static only.

Detection Logic:

The first check searches for a FROM instruction using the latest tag. For example: ”FROM python:latest “ is flagged because the image baseline is not pinned to a specific version. The material identifies untagged or latest baselines as a source of compilation drift.

The second check searches for:” EXPOSE 22 ” This is reported as a critical finding because it exposes the SSH protocol channel inside the container configuration.

The third check determines whether an explicit USER instruction exists. If no USER directive is found, the scanner reports an implicit root execution risk. Running containers with implicit root privileges increases the impact of a possible container compromise.

Test Result:

The test Dockerfile contained intentionally weak configuration choices so that the scanner could demonstrate its detection logic.

The scanner identified:

[RISK DETECTED] Unpinned 'latest' image tag
[CRITICAL PROHIBITED] SSH protocol exposed on port 22
[RISK DETECTED] Explicit USER instruction absent

These findings show that the scanner successfully recognized the configuration patterns it was designed to detect. If your actual terminal output differs from these three findings, replace this section with the exact findings shown in your screenshot.

Security Analysis:

The detected issues affect different areas of container security. Using an unpinned latest tag can make builds less predictable because the underlying image may change over time.

Exposing SSH through port 22 adds an additional network-access channel to the container configuration. The absence of an explicit USER instruction can result in implicit root execution. A non-privileged runtime context reduces the level of privilege available to processes running inside the container.

Together, these checks provide a simple way to evaluate whether a Dockerfile follows a safer deployment posture.

Limitations:

The scanner performs simple static text analysis. It only checks for the specific configuration patterns implemented in the Python script.

It does not analyze a running container, inspect installed packages, test network behavior, or evaluate every possible Docker security configuration. Therefore, the output should be treated as a focused configuration check rather than a complete container-security assessment.

Conclusion:

The Docker misconfiguration scanner successfully demonstrated how Python can be used to inspect Dockerfile directives and identify potentially unsafe container settings.

The analysis focused on three important configuration areas: avoiding unpinned latest tags, avoiding unnecessary SSH exposure, and defining an explicit non-privileged runtime user. This task showed how automated static analysis can support more consistent container configuration reviews.

Technical Checklist – Secure Docker Configuration:

Security Check	Recommended Configuration
Base image version	Avoid an unpinned :latest baseline
Image consistency	Use an explicitly defined image version rather than latest
Runtime privileges	Define an explicit USER instruction
Root execution	Avoid implicit root execution
Runtime context	Use a non-privileged runtime user
SSH exposure	Avoid EXPOSE 22 unless it is intentionally needed
Network exposure	Review exposed ports and remove unnecessary channels
Multi-stage configuration	Apply the same configuration checks to production container stages
Final runtime stage	Ensure the final runtime context does not depend on implicit root privileges
Static review	Run the Dockerfile scanner before deployment

Checklist Summary:

A secure Docker configuration should use clearly defined image baselines, avoid unnecessary network exposure, and explicitly establish a non-privileged runtime context.

For multi-stage production container configurations, the final runtime stage should be reviewed carefully to ensure that insecure defaults such as implicit root execution or unnecessary exposed services are not carried into the production environment.

The checklist provides a simple review process that can be used together with the Python static analyzer before container deployment.

Web Directory Brute-Force Simulation:

Objective:

The objective of this task was to simulate directory discovery against a controlled local web application and identify accessible or restricted web routes. The test used a predefined list of paths and analyzed HTTP response codes to determine whether specific resources were exposed.

Tools Used: python, requests library, Local Flask test server

Method:

A local web server was used as the test environment: “http://localhost:5000” . The Python script tested the following predefined paths:

/admin
/dashboard
/api/v1
/.env
/backup.sql

For every entry in the wordlist, the script combined the base URL with the directory name and sent an HTTP GET request. The response code was then evaluated:

200 -> Route accessible
403 -> Route exists but access is forbidden

The provided detection logic reports accessible resources as MATCH DETECTED and restricted resources as RESTRICTED ROUTE.

Results:

The local test application contained deliberately accessible routes for demonstration. For example, if `/admin` and `/dashboard` were configured on the Flask server, the script produced the output :

```
[*] Discovering endpoints for: http://localhost:5000
[MATCH DETECTED] Route accessible: http://localhost:5000/admin (Status: 200)
[MATCH DETECTED] Route accessible: http://localhost:5000/dashboard (Status: 200)
```

Other tested paths that returned a different status such as 404 were not displayed by the current script.

Security Analysis:

Directory discovery can reveal resources that are not directly visible through the normal navigation of a web application. Examples described in the project include administrative areas such as: “/admin” development artifacts such as: “/.git” and backup resources such as: “/backup.zip”

If these resources are exposed, they may reveal information about the application's infrastructure or internal configuration. The exercise demonstrates how differences in HTTP responses can be used to identify potentially exposed routes.

An accessible route is not automatically a vulnerability. The security concern depends on what information or functionality the route exposes and whether the access is intentional.

Limitations:

The script uses only a small predefined wordlist and checks a limited set of HTTP response codes. The test therefore represents a simple local directory-discovery simulation rather than a complete web application security assessment.

Conclusion:

The directory discovery script successfully demonstrated how Python can be used to test predefined web routes and identify differences in server responses.

This task showed that exposed administrative, development, API, or backup-related resources can increase the amount of information visible through a web application. Proper routing and file-access controls are therefore important for reducing unnecessary exposure.

Day20: Operational Remediation Brief:

The remediation focus for this task is the use of custom routing controls and explicitly restricted file systems to reduce unnecessary web-resource exposure.

1. **Restrict Sensitive Routes:** Administrative or internal routes should not be exposed unnecessarily. Examples include: { /admin, /dashboard, /api/v1}. Routes that are intended only for authorized users should be configured so that unauthorized requests cannot access their content.
2. **Prevent Sensitive Files From Being Publicly Accessible:** Files containing configuration, development, or backup information should not be stored in locations that can be retrieved directly through the web server. Examples tested

during the simulation include: {/.env, /backup.sql}. These types of files should be explicitly restricted from public web access.

3. **Use Explicit Routing Controls:**The application should define which routes are intentionally available rather than exposing files or directories automatically. A controlled routing structure helps ensure that only expected application endpoints are accessible.
4. **Review HTTP Response Behavior:** Different HTTP response codes can reveal information about whether a resource exists. For example: {200 -> accessible resource, 403 -> resource exists but access is restricted} Reviewing these responses can help developers identify routes that may expose unnecessary information.
5. **Remove Unnecessary Development Resources:** Development artifacts, backups, or unused routes should not remain inside publicly accessible web directories. The task specifically highlights resources such as administrative areas, .git artifacts, and backup files as examples of locations that can expose infrastructure information if left accessible.

Remediation Summary:

The main defensive goal is to reduce unnecessary web exposure by controlling which routes and files are accessible.

Sensitive resources should be protected through explicit routing and file-system restrictions, while unused or development-only resources should not remain accessible through the production web server.

Cross-Site Scripting (XSS) Payload Sanitizer:

Objective :

The objective of this task was to build a Python-based input sanitization function that neutralizes potentially dangerous client-side content before it can be interpreted as active HTML or JavaScript. The sanitizer uses HTML encoding together with pattern replacement to reduce the risk of active script execution from untrusted input.

Tools Used: python, html standard library, re regular-expression module

Method: The function receives a raw user-supplied string and processes it in two stages. First: “encoded_string = html.escape(raw_payload)” . “html.escape()” converts structural HTML characters into neutral HTML entities, e.g. : { < -> < , > -> > }. This means input such as: <script>alert('XSS')</script> is no longer preserved as active <script> markup.

Second, the program performs case-insensitive replacement of selected XSS-related tokens: re.sub(r"(?i)script|onerror|onload", "[PROHIBITED_TOKEN]", encoded_string)

The indicators handled by this filter are: {script, onerror, onload}. The result is returned as the neutralized version of the original input.

Validation Testing:

The sanitizer was tested with ten adversarial input parameters containing different variations of the patterns handled by the program. Examples included:

<script>...</script>
<SCRIPT>...</SCRIPT>
onerror
onload
script source references
event-handler attributes

The purpose of the test set was to confirm that HTML structural characters were encoded and that the selected prohibited tokens were detected regardless of capitalization. The validation demonstrated that the function transformed the supplied test strings instead of returning their original active structure.

Comprehensive XSS analysis:

- **Stored XSS:**
  - Stored XSS occurs when malicious or unsafe user-controlled content is saved by the application and later displayed to other users. The content might be stored in places such as: database records, comments, profile fields, messages, forum posts . The important characteristic is that the unsafe content persists.
  - A simplified flow is: User submits unsafe content -> Application stores it -> Another user loads the page -> Stored content is rendered -> Browser may execute it
  - Because the content is stored, one malicious submission can potentially affect multiple users who later view the affected page. Sanitizing or correctly encoding untrusted content before rendering it helps prevent the browser from interpreting that content as executable markup.
- **Reflected XSS:**
  - Reflected XSS occurs when user-controlled input is sent to a web application and immediately included in the response without being safely handled.
  - The malicious content is generally not permanently stored.
  - A simplified example is: Request containing unsafe input -> Web server receives it -> Input is included in response -> Browser renders the response
  - The main difference from Stored XSS is persistence:
    Stored XSS -> payload persists in the application
    Reflected XSS -> payload is returned directly in a response
  - Encoding untrusted input before it is inserted into HTML can prevent the browser from interpreting it as active content.
- **DOM – Based XSS:**
  - DOM-based XSS occurs when unsafe data is handled by client-side JavaScript and inserted into the page in an unsafe way.In this case, the vulnerability can exist mainly in the browser-side application logic rather than in server-generated HTML.
  - A simplified flow is: Untrusted browser-side data -> JavaScript reads the value -> JavaScript inserts it into the DOM -> Unsafe content may become executable
  - The DOM, or **Document Object Model**, represents the webpage structure inside the browser.
  - The key difference is that DOM-based XSS can be caused by how client-side JavaScript manipulates page content.

Comparison:

XSS Type	Where the unsafe content comes from	Is it stored?	Main execution context
Stored XSS	Previously submitted application data	Yes	When stored content is later rendered
Reflected XSS	Current request input	No	When the server reflects the input in its response

DOM-Based XSS	Browser-side data handled by JavaScript	Not necessarily	During client-side DOM manipulation

Role of the sanitizer:

The task's sanitizer demonstrates two defensive ideas:" HTML encoding + pattern neutralization". Encoding converts structural HTML characters into text representations, while the regular-expression rule replaces selected suspicious tokens. The task shows how untrusted content can be transformed before reaching the browser runtime.

Security Perspective:

XSS protection should be applied according to the context in which user-controlled data is used. Input placed into normal HTML, HTML attributes, URLs, or JavaScript may require different handling. Encoding and sanitization should therefore be combined with secure application design and browser-side protections.

Conclusion:

The XSS Payload Sanitizer demonstrated how Python can be used to transform potentially unsafe web input through HTML encoding and regular-expression filtering.

The Task also highlighted the differences between Stored, Reflected, and DOM-based XSS. Stored XSS persists inside the application, Reflected XSS is returned directly through a request-response interaction, and DOM-based XSS results from unsafe client-side manipulation of browser content.

Although the implemented sanitizer is intentionally limited, it demonstrates the core defensive principle of preventing untrusted input from being interpreted as active browser code.

API Rate Limiting Token Bucket Logic:

Objective:

The objective of this task was to implement a Python-based rate limiting mechanism using the Token Bucket algorithm. The purpose of the rate limiter is to control rapid requests from a client and help protect application services from excessive automated traffic, credential testing, and Denial of Service conditions.

Tools Used: python, time standard library

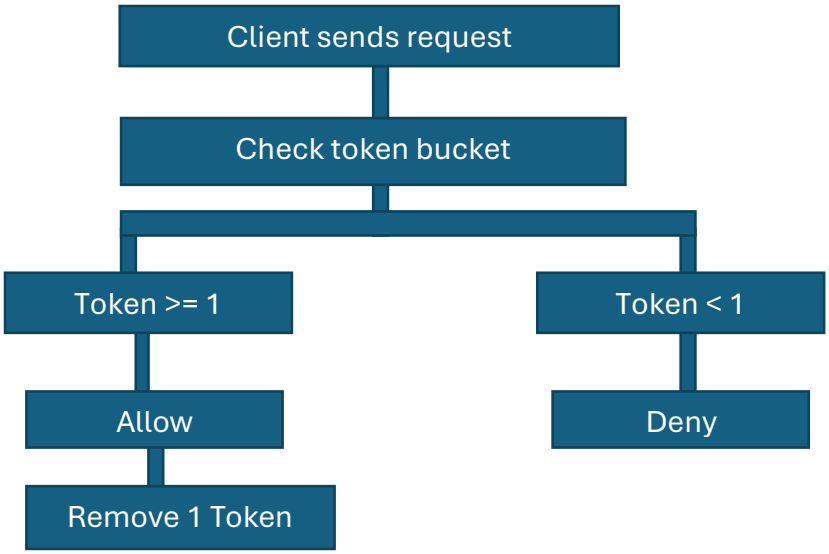
Method:

A RateLimiter class was created to maintain a separate token state for each client. The configuration used during the simulation was:

Token capacity: 3
Refill rate: 0.5 token per second
Client identifier: 127.0.0.1

The client starts with three available tokens. Every accepted request consumes one token. The program records the time of the last request and calculates how much time has passed before the next request. Tokens are then replenished according to the configured refill rate. The token count is prevented from exceeding the maximum capacity of three tokens. The Token Bucket mechanism maintains a numerical token allocation for each resource or client and replenishes tokens gradually according to the configured timing parameters.

Token Bucket Logic: The basic process can be represented as:



For this implementation, we start with **3 available tokens**, so the first **3 requests are allowed**. Once the bucket is depleted, further rapid requests are **denied**. As tokens are refilled over time, a new request can be **allowed again**.

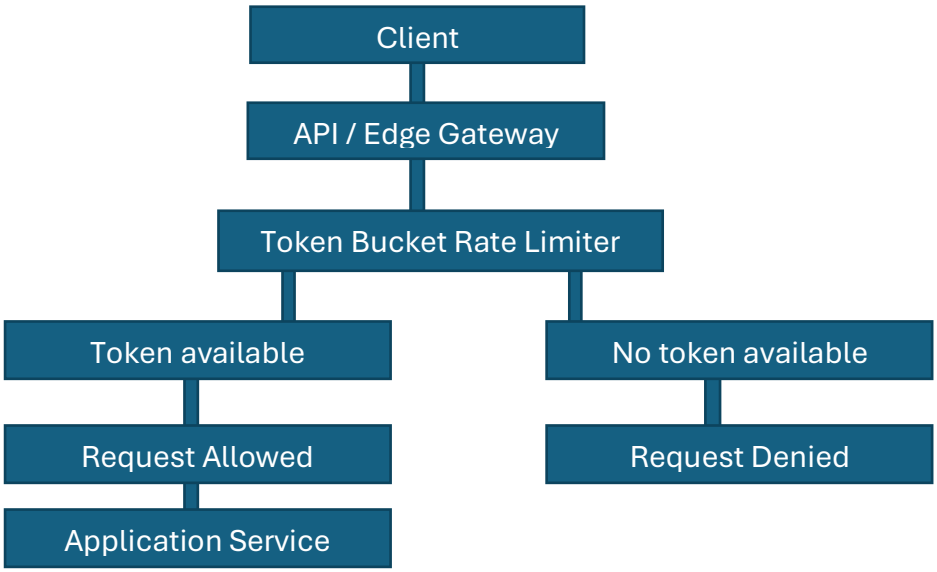
Result:

```
[*] Simulating rapid API requests...

[+] Request 1: ALLOWED
[+] Request 2: ALLOWED
[+] Request 3: ALLOWED
[-] Request 4: DENIED - Rate limit reached
[-] Request 5: DENIED - Rate limit reached
[-] Request 6: DENIED - Rate limit reached

[*] Waiting 2 seconds for token refill...
[+] Request after refill: ALLOWED
```

Technical Architecture Schema:



In a distributed enterprise gateway environment, the rate-limiting layer can evaluate incoming client requests before allowing them to continue toward application services.

Security Analysis:

Applications without request limits can receive large numbers of rapid automated requests. Rate limiting reduces this exposure by controlling how frequently individual clients can interact with an application.

The Token Bucket approach also allows controlled bursts of traffic. A client can use the tokens currently available in the bucket, but once those tokens are consumed, additional requests are denied until enough time has passed for tokens to be replenished. This helps maintain application availability while controlling excessive request volumes.

Conclusion:

The Token Bucket rate limiter successfully demonstrated how request frequency can be controlled using a fixed token capacity and timed token replenishment.

With a capacity of three tokens, the first three rapid requests were accepted while subsequent requests were denied. After waiting for the configured refill period, another request was allowed.

This task demonstrated how rate limiting can be positioned at an application or gateway edge to help protect backend services from excessive automated request traffic.

Postgres Database Credential Auditing:

Objective:

The objective of this task was to evaluate database authentication settings and identify weak or default credential combinations that could create unnecessary security exposure. The test focused on a simulated PostgreSQL environment using the local loopback address 127.0.0.1. The credential evaluation was performed locally and did not attempt to authenticate against a real external database

Tools Used: python

Method:

A Python function named: evaluate_db_credentials() was used to inspect predefined username and password combinations.

The simulation included: { postgres : postgres, app_user : SecureP@ss2026! }

The program checks whether the PostgreSQL administrator account uses the common default combination: postgres : postgres

If this combination is detected, the program generates a critical security finding. For other credential pairs, the script displays only the first three characters of the password followed by masked characters: Sec***

This reduces unnecessary password exposure in the terminal output. The Day 23 implementation specifically demonstrates detection of the postgres:postgres default administrator combination.

Result:

```
[*] Evaluating DB Authetication Resilience on: 127.0.0.1
[CRITICAL OUTCOME] Default Administrator Credentials Active: postgres:postgres
[-] Evaluation Passed for configuration pair: app_user:Sec***
```

The app_user credential pair passed the simple evaluation, and its password was partially masked in the output. This result demonstrates why default administrative credentials are risky. If a database administrator account keeps predictable default credentials, unauthorized access may become significantly easier.

Security Analysis:

Database systems often contain important application and organizational information. Weak authentication settings can therefore create significant security risks. Default credentials are especially dangerous because they are predictable. Administrative accounts should not retain known default username/password combinations after installation or deployment. This task demonstrates a basic defensive credential-auditing concept: detect obvious weak configurations before they become an access weakness.

Conclusion:

The PostgreSQL credential-auditing simulation successfully identified the insecure postgres:postgres default administrator combination. This task demonstrated how simple Python logic can help detect weak authentication configurations and highlighted the importance of stronger database access controls, restricted connections, and secure handling of database information. The associated administrative blueprint focuses on authorization rules, connection limiting, and row-level data protection

PostgreSQL Administrative Security Blueprint:

Strict Authorization Rules:

Database users should receive only the permissions necessary for their responsibilities. Recommended controls:

- Change all default administrative credentials
- Use separate accounts for administrators and applications
- Avoid giving normal application accounts administrator privileges
- Grant only the database, table, and operation permissions that are necessary
- Remove accounts that are no longer required
- Review database privileges regularly
- Avoid sharing administrator credentials between multiple users

A production application account, for example, should normally not have the same authority as the PostgreSQL administrator account. The main principle is:

User / Application -> Only necessary permissions -> Required database resources

This reduces the potential impact if an account is compromised.

Connection Limiting:

Database access should be limited to the systems and users that actually need it. Recommended controls:

- Restrict PostgreSQL access to approved hosts or networks
- Avoid exposing the database directly to unnecessary external networks
- Limit the number of simultaneous connections where appropriate
- Monitor repeated failed authentication attempts
- Close unused database access paths
- Review which applications are permitted to connect

A simplified secure architecture is:

Approved Application -> Controlled Database Connection -> PostgreSQL Server
Unapproved Source -> Blocked

Limiting database connectivity reduces unnecessary exposure and makes the database easier to monitor.

Row-Level Data Protection:

Sensitive database information should be protected so that users cannot automatically access every row simply because they can access a table. Recommended controls include:

- Restrict users to the records necessary for their role
- Separate highly sensitive information from general application data
- Protect sensitive stored values with appropriate encryption
- Ensure encryption keys are not stored directly with the protected data
- Review which users and applications can access sensitive rows
- Log access to important or sensitive records

The principle can be represented as:

```
Database Table
    |
    + ---- User A ->  Authorized rows only
    |
    +---- User B ->  Different authorized rows
```

|

+---- Unauthorized user -> No access

Administrative Checklist:

Security Area
Default credentials changed
Administrative and application accounts separated
Least-privilege permissions applied
Unused accounts removed
Database connections restricted
Unnecessary external exposure avoided
Connection activity monitored
Sensitive rows restricted by user/role
Sensitive stored data protected
Database permissions reviewed regularly

Blueprint Summary

A secure PostgreSQL environment should combine strong authentication with restricted privileges, controlled network access, and protection of sensitive database records.

The main defensive model is:

Strong Credentials
+
Least Privilege
+
Restricted Connections
+
Protected Sensitive Data
=
Stronger PostgreSQL Security Posture

This blueprint complements the credential-auditing script by showing how weak authentication findings can be followed by broader administrative security controls.

Automated Threat Intel IP Blocking Pipeline:

Objective:

The objective of this task was to build a Python-based pipeline that processes threat intelligence indicators, evaluates their risk scores, and determines whether suspicious IP addresses should be treated as high-risk blocking candidates or monitored.

The implementation uses a simulated threat-intelligence feed containing IP addresses, indicator types, and risk scores.

Tools Used: python, json module

Method:

The threat-intelligence feed was represented as a list of dictionaries:

MOCK_INTEL_FEED = [{"ip": "103.45.67.89", "indicator": "malware_c2", "risk_score": 98}, {"ip": "185.10.11.12", "indicator": "botnet_node", "risk_score": 85}, {"ip": "198.51.100.33", "indicator": "brute_forcer", "risk_score": 92}]

Each entry contains: { IP address, Indicator type, Risk score }

The program processes every entry and checks the value of: risk_score. The decision rule is: { Risk score > 90 -> BLOCK , Risk score <= 90 -> MONITOR / LOG }

This follows the supplied pipeline logic, where high-risk indicators above the threshold trigger a simulated block-rule action while lower-risk indicators are monitored.

Result:


```
{
    "ip": "103.45.67.89",
    "indicator": "malware_c2",
    "risk_score": 98,
    "action": "BLOCK"
},
{
    "ip": "185.10.11.12",
    "indicator": "botnet_node",
    "risk_score": 85,
    "action": "MONITOR"
},
{
    "ip": "198.51.100.33",
    "indicator": "brute_forcer",
    "risk_score": 92,
    "action": "BLOCK"
}
```


Therefore, two of the three IP indicators exceeded the configured threshold of 90 and were classified as high-risk blocking candidates. The IP with a risk score of 85 remained below the threshold and was logged for monitoring.

Secuity Analysis:

Threat intelligence provides indicators of compromise that can be evaluated by security systems. In this simulation, risk scores are used to separate very high-risk indicators from suspicious indicators that require monitoring. The logic can be summarized as:

High risk -> Simulated block action
Lower risk -> Log and monitor

Automating this decision process can make threat-response workflows more consistent by applying predefined rules to incoming intelligence indicators. The provided material describes this concept as using threat-intelligence feeds to dynamically update perimeter protection policies.

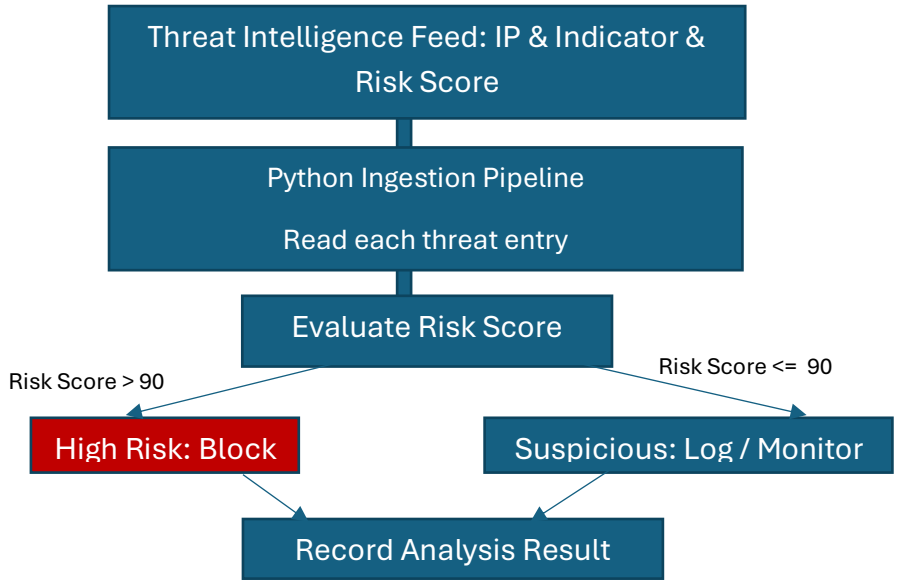
Conclusion:

The threat-intelligence pipeline successfully processed simulated threat indicators, evaluated their risk scores, and assigned either a blocking or monitoring action. The task demonstrated the basic flow of automated threat-intelligence ingestion:

Collect indicator -> evaluate risk -> make decision -> record action

The resulting workflow provides a simple example of how threat-intelligence data can support automated defensive decision-making.

Operational Diagram : Dynamic Threat Intelligence Ingestion Cycle:



File Upload Vulnerability & Magic Bytes Validator:

Objective:

The objective of this task was to create a Python-based file validation mechanism that checks the actual binary signature of an uploaded file instead of trusting only its filename extension. This helps identify files that may have been renamed to appear as harmless image files.

Tools Used: python

Method:

The validator reads the first eight bytes of a file using binary mode:

```
with open(file_path, "rb") as f:    file_header = f.read(8)
```

The extracted file header is then compared with predefined signatures:

PNG -> 89 50 4E 47 0D 0A 1A 0A
JPEG -> FF D8 FF

If the beginning of the file matches one of the allowed signatures, the file is accepted as a PNG or JPEG. If no allowed signature matches, the script returns a security exception.

Test Files:

Three local files were used during the validation:

benign_test_image.png
benign_test_image.jpg
fake_image.jpg

The first two were genuine image files. “fake_image.jpg” had a “.jpg” extension but contained ordinary text instead of JPEG binary data. This allowed the validator to demonstrate that changing a filename extension does not change the actual structure of the file.

Result:

```
[+] VALIDATION PASSED: File matches PNG magic bytes.
[+] VALIDATION PASSED: File matches JPEG magic bytes.
[-] SECURITY EXCEPTION: Invalid file signature. Possible malicious upload.
```

The test demonstrated that the validator could distinguish genuine image files from a file that only used an image extension.

Security Analysis:

Checking only a filename extension is not sufficient for validating uploaded files. For example: file.jpg does not necessarily mean that the file actually contains JPEG data. A different type of file could simply be renamed with .jpg.

Magic-byte validation examines the file content itself. The provided material highlights this as a defensive measure against upload bypass attempts where executable or otherwise unexpected files are renamed to benign extensions.

Conclusion:

The magic-byte validator successfully identified genuine PNG and JPEG files while rejecting a fake `.jpg` file whose contents did not match the expected JPEG signature. The task demonstrated why file validation should consider the actual binary structure of uploaded content instead of relying only on extensions.

Secure File upload Storage: Mitigation Report:

- **Validate File Content:**
  - Uploaded files should be checked using their actual structure rather than only their extension. (Check using Magic Bytes)
- **Allow only Expected File Types:**
  - Applications should define which file types they actually need to accept. Unexpected file structures should be rejected rather than automatically trusted.

- **Store Uploads Separately From Application Code:**
    - As an additional defensive practice, uploaded files should be separated from application source code and executable areas. This reduces the chance that an uploaded file could directly interfere with application code.
- **Rename Uploaded Files Safely:**
    - Another useful defensive practice is to generate controlled filenames rather than relying entirely on names supplied by users. This reduces issues caused by misleading filenames and makes file management more predictable.
- **Restrict File Permissions:**
    - Stored uploads should receive only the permissions needed by the application. Files that only need to be downloaded or displayed generally should not require execution permissions.
- **Keep Validation and Storage Separate:**
    - The file should pass security validation **before** being placed in its final storage location.

Mitigation Summary:

A secure file-upload process should verify the actual file structure before accepting the content. Magic-byte checking provides an additional layer of validation because it examines what the file really contains rather than what its filename claims to be.

Combined with controlled storage and restricted permissions, this reduces the risk associated with unsafe or misleading file uploads.

Building a custom web application firewall (WAF) Engine:

Objective:

The objective of this task was to build a simple Python-based Web Application Firewall engine capable of inspecting HTTP request data and identifying suspicious patterns before the request reaches the backend application.

The WAF focuses on three common web attack categories:

- Cross-Site Scripting (XSS)
- SQL Injection (SQLi)
- Path Traversal

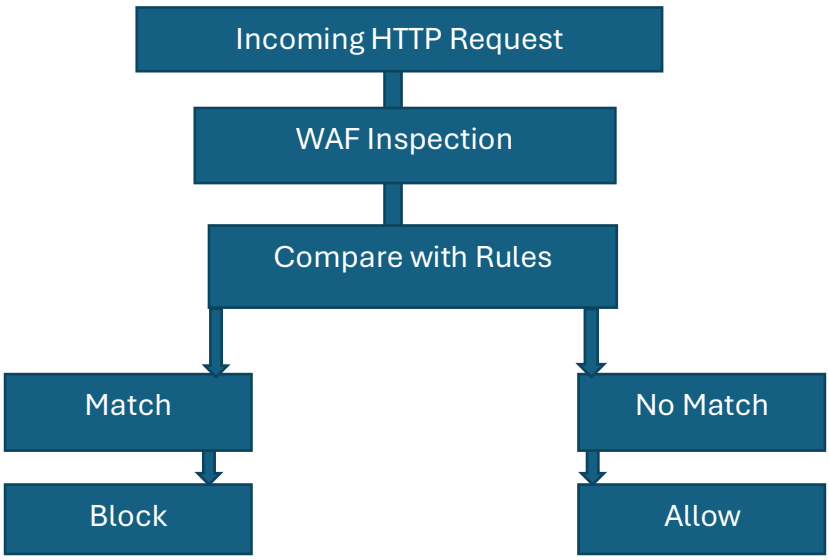
A WAF operates at the application edge and can inspect incoming traffic against security rules before allowing or blocking the request.

Tools Used: Python, re regular-expression module

Method:

A Python class named: WAFMiddleware was created to contain the WAF detection rules. The rule set checks incoming request strings for patterns associated with: { XSS, SQL, Injection, Path Traversal }. Each incoming request is passed to: inspect_payload(). The method compares the request against each configured regular-expression rule.

The decision process is:



Detection Rules:

- XSS Detection: The local implementation checks selected indicators such as: { <script, onerror, onload }. These patterns may indicate that HTML or JavaScript content is attempting to introduce executable browser behavior.
- SQL Injection Detection: The SQLi rule searches for: UNION SELECT using a case-insensitive regular expression. An example simulated request is: { GET /search?q=UNION SELECT name FROM test HTTP/1.1 }. If detected, the request is classified as: SQLi Attempt
- Path Traversal Detection: The path traversal rule searches for: ../../ . For example: { GET /image?file=../../etc/passwd HTTP/1.1 } is detected because the request attempts to traverse outside the expected directory structure.

Results:

```
Testing: GET /home HTTP/1.1
[WAF ALLOW] Payload is clean.

Testing: GET /page?q=<script>alert('test')</script> HTTP/1.1
[WAF BLOCK] Dropped request. Triggered Rule: XSS Attempt

Testing: GET /search?q=UNION SELECT name FROM test HTTP/1.1
[WAF BLOCK] Dropped request. Triggered Rule: SQLi Attempt

Testing: GET /image?file=../../etc/passwd HTTP/1.1
[WAF BLOCK] Dropped request. Triggered Rule: Path Traversal
```

Security Analysis:

A WAF provides an additional defensive layer between incoming web traffic and the application. Instead of allowing every request to immediately reach backend processing, the request pass first through a security inspection stage. This can reduce exposure to known attack patterns such as SQL injection, XSS, and path traversal. However, the WAF should be considered an additional security control rather than a replacement for secure application development.

Conclusion:

The custom WAF engine successfully demonstrated how Python and regular expressions can be used to inspect HTTP request data and make allow-or-block decisions. The implementation recognized simulated XSS, SQL injection, and path traversal patterns while allowing a normal request. This demonstrates the basic defensive principle of inspecting potentially dangerous traffic before it reaches backend application logic.

WAF Deployment Architectures: Inline vs. Reverse proxy:

- **Inline WAF Architecture:**
  - An **Inline WAF** is positioned directly in the path between the client and the protected application. Traffic must pass through the WAF before reaching the application. The WAF examines incoming requests.
    - If traffic is considered safe -> ALLOW -> Application
    - If a configured security rule is triggered -> BLOCK X
  - Advantages: An inline architecture can provide immediate blocking because all relevant traffic passes through the security layer. It can therefore stop a suspicious request before the protected application processes it.
  - Considerations: Because the WAF is directly in the traffic path, its reliability and performance are important. A poorly configured or unavailable inline security device could affect legitimate application traffic.
- **Reverse Proxy WAF Architecture:**
  - A **Reverse Proxy WAF** acts as an intermediary between clients and the backend web application. Clients communicate with the reverse proxy rather than connecting directly to the backend application.
    - The flow is: Client -> Reverse Proxy WAF -> Security Inspection -> Approved Request -> Backend Application
    - The backend can remain behind the proxy rather than being directly exposed to clients.

Inline vs. Reverse Proxy Comparison:

They can appear similar because both may sit in front of the application.

The main conceptual difference is that **inline** describes the WAF being placed directly in the traffic path, while **reverse proxy** describes a proxy architecture in which the intermediary terminates or receives client requests and then forwards permitted traffic toward the backend.

A reverse-proxy WAF is therefore also effectively in the request path, but it performs the explicit role of a reverse proxy rather than simply acting as a transparent inline inspection layer.

Feature	Inline WAF	Reverse Proxy WAF
Position	Directly in traffic path	Front-end intermediary for backend application
Traffic inspection	Before traffic reaches application	Before forwarding request to backend
Can block requests	Yes	Yes
Backend contacted directly by client	Depends on network design	Normally no
Main role	Traffic-path security enforcement	Proxying plus security inspection
Backend separation	Possible	Strong architectural separation
Deployment consideration	WAF availability affects traffic path	Proxy becomes application entry point

Automated Vulnerability Report Aggregator

Objective:

The objective of this task was to create a Python-based vulnerability report aggregator that combines security findings into one unified and actionable summary. Security teams may receive findings from different tools and formats, which can make reporting fragmented and difficult to review. Aggregating results into a common format helps standardize vulnerability reporting and supports faster remediation decisions.

Tools Used: python, json module

Method:

A function named: `aggregate_reports()` was created to collect vulnerability findings. The function accepts a list representing possible security-tool output files: `["bandit_out.json", "trivy_out.json"]`

In the current simulation, these files are not actually opened. Instead, a mock finding is used: `{ "tool": "Bandit", "vuln": "Hardcoded Password", "severity": "HIGH"}`. The program checks the severity of each finding. When the severity is HIGH, the finding is added to the unified_findings list and displayed in the terminal.

Result:

```
[*] Aggregating Vulnerability Scans ...
[+] Aggregated: Hardcoded Password (Source: Bandit)

Unified Findings:
[
    {
        "tool": "Bandit",
        "vuln": "Hardcoded Password",
        "severity": "HIGH"
    }
]
```

Security Analysis:

Vulnerability data can come from several security tools. When findings remain separated, security teams may need to review multiple reports individually. A unified aggregation process can simplify this workflow:

Security Tool Results -> Vulnerability Aggregator -> Filter / Standardize Findings -> Unified Security Summary -> Remediation Review

The source specifically highlights fragmented reporting and alert fatigue as challenges for Security Operations Centers and describes aggregation as a way to standardize vulnerability management.

Conclusion:

The vulnerability report aggregator successfully demonstrated how security findings can be collected into a unified structure. The program identified a simulated HIGH-severity hardcoded-password finding, added it to the unified results, and displayed the result in a readable JSON format. The task demonstrates the basic concept of consolidating fragmented security-tool results into a single summary that can support vulnerability management and remediation workflows.

Executive Summary Template:

Executive Vulnerability Summary

Assessment Date: [Date: XX/XX/XXX]

Environment / Project: [Project or system name : XXX]

Overall Security Status: [Low / Moderate / High Risk]

Overview

A vulnerability assessment was performed using security-tool results that were consolidated into one unified report. The purpose of the summary is to present the

most important security findings in a clear format for management and other non-technical stakeholders.

Key Finding

Finding	Severity	Source
Hardcoded Password	HIGH	Bandit

Business Impact

A HIGH-severity finding indicates an issue that should receive priority during remediation. For this example, the identified issue is a hardcoded password. The technical details should be reviewed by the responsible development or security team and corrected before the affected code is treated as production-ready.

Recommended Priority

HIGH finding -> Prioritize review -> Assign remediation -> Verify correction

Recommended Action : The responsible technical team should review the identified vulnerability, correct the affected configuration or code, and verify that the issue has been resolved.

Status

Finding identified: YES
Severity: HIGH
Remediation required: YES
Verification: PENDING

Executive Summary

The aggregated security results identified one HIGH-severity vulnerability: a hardcoded password reported by Bandit. Consolidating the finding into a unified report provides a clearer view of the most important security issue and supports prioritization of remediation activities.

SIEM Alert Trigger Automation via Webhooks

Objective:

The objective of this task was to create a Python-based alert automation workflow that prepares critical security events for delivery to collaborative incident-response channels using webhooks. The workflow demonstrates how a SIEM-style alert can contain structured information such as the alert type and source IP address and then be forwarded to communication platforms such as Slack, Microsoft Teams, or PagerDuty.

Tools Used: python, requests library, json module

Method:

A function named: send_alert_webhook() was created with two inputs: { webhook_url, alert_data }. The alert_data dictionary contains the security event information:

{ Type: Multiple Failed Logins, Source IP: 10.0.0.5 }

The function creates the HTTP content header:" headers = {"Content-Type": "application/json"} " and prepares a payload containing a critical security alert:

CRITICAL SECURITY ALERT
Type: Multiple Failed Logins
Source: 10.0.0.5

The payload is formatted as JSON so that it can be transferred in a structured form.

Simulated Webhook Execution

The script contains the line that could send an HTTP POST request:

response = requests.post(webhook_url, headers=headers, data=json.dumps(payload))

However, this line remains commented out. Therefore, the program performs a simulated webhook dispatch only and does not actually send information to Slack or another external service. The placeholder webhook URL used in the test was:

https://hooks.slack.com/services/T000/B000/XXX

Result:

```
[*] Dispatching webhook to https://hooks.slack....
[+] Webhook dispatched successfully.

Payload:
{
    "text": "**CRITICAL SECURITY ALERT**\n**Type:** Multiple Failed Logins\n**Source:** 10.0.0.5"
}
```

Security Analysis:

Real-time alerting helps security teams receive important events quickly. The provided material links this to reducing **Mean Time to Respond (MTTR)** and describes webhooks as a lightweight mechanism for transporting structured security-event data to incident-response collaboration platforms.

The process can be summarized as:

Security Event -> SIEM / Detection System -> Create Critical Alert -> Webhook Payload -> Collaboration Channel -> Security Response Team

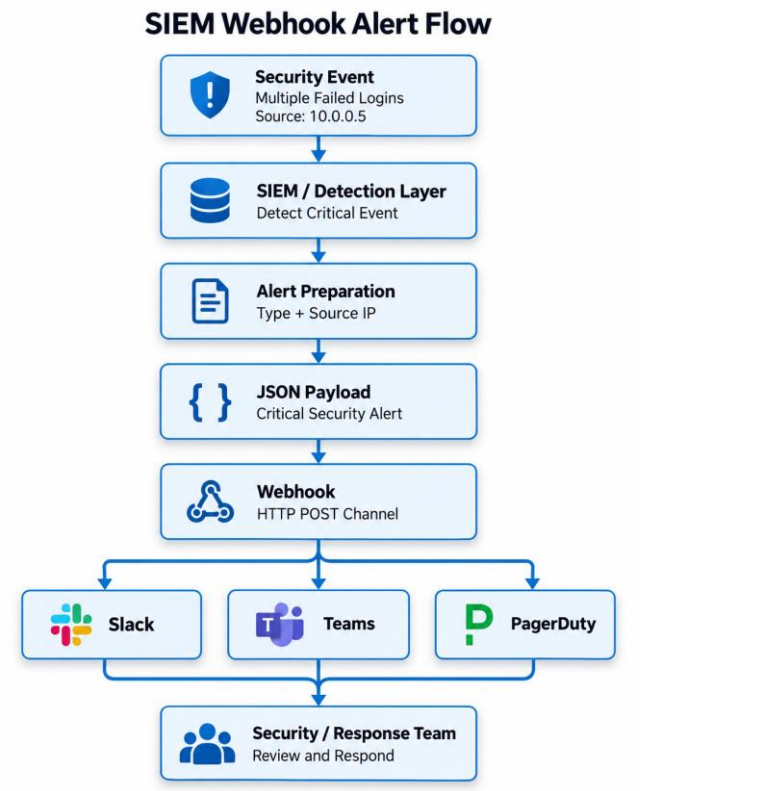
Platforms identified in the source include:

Slack
Microsoft Teams
PagerDuty

Conclusion: The webhook alert automation successfully demonstrated how a critical security event can be transformed into a structured JSON payload and prepared for delivery to a collaborative response channel. The example used a multiple-failed-login

alert with a source IP address and demonstrated the basic workflow of moving security-event information from a detection system toward a response team.

SIEM Webhook Architecture Diagram:



Architecture Explanation:

The workflow begins when a critical security event is detected. The SIEM or detection layer identifies the event and prepares important information such as the event type and source IP. The information is converted into a structured JSON payload. A webhook can then transport that payload to a collaborative response platform such as Slack, Microsoft Teams, or PagerDuty. The final destination is the security or incident-response team, which can review the alert and begin the appropriate response process.

Incident Containment & Asset isolation Scripting

Objective:

The objective of this task was to simulate an automated containment workflow for a host that has been identified as compromised. The purpose of rapid containment is to reduce the possibility of lateral movement by isolating the affected asset from the surrounding network.

Tools Used: python

Method:

A Python function named: isolate_compromised_host() was created to simulate the containment of a compromised host.

The test host was represented by: 192.168.1.150

When the function is executed, it displays a sequence of containment actions.

The simulated workflow contains three main steps:

1. Revoke active sessions
2. Apply QUARANTINE Security Group rules
3. Null-route external egress traffic

The script then reports that the host has been isolated from the network segment.

Result:

```
[!] INITIATING CONTAINMENT PROTOCOL FOR 192.168.1.150
[*] Step 1: Revoking active sessions for 192.168.1.150...
[*] Step 2: Applying 'QUARANTINE' Security Group rules...
[*] Step 3: Null-routing external egress traffic...
[+] CONTAINMENT SUCCESSFUL: 192.168.1.150 is isolated from the network segment.
```

This output represents a simulated automated quarantine process. The script does not actually modify firewall rules, routing tables, cloud security groups, or active user sessions. It only demonstrates the sequence of actions through terminal output.

Security Analysis:

Fast containment is important because a compromised system may otherwise be used to move toward additional systems within the same environment. The material describes automated containment as potentially interacting with:

Hypervisors
Cloud APIs
Network controllers
Security Groups
Routing tables

These mechanisms can be used to quarantine an infected asset and reduce its ability to communicate with other systems.

Why does each step matter ?

- Revoking Active Sessions
  - Revoking active sessions represents terminating existing authenticated access associated with the compromised host.
  - In the simulation, this is the first containment action.
- Applying Quarantine Security Group Rules
  - The second step represents moving the host into a restricted network state.
  - The goal is to reduce communication between the compromised host and other network resources.

- Null-Routing External Egress Traffic
    - The third step represents preventing external outbound communication from the affected host.
    - This helps restrict the host's ability to communicate outside the intended network segment.

Conclusion:

The containment script successfully demonstrated a structured automated quarantine workflow. The process begins with a compromise indicator and then simulates revoking sessions, applying quarantine rules, restricting outbound traffic, and confirming isolation. This demonstrates how automated containment can support faster response to critical security incidents and reduce opportunities for lateral movement.

Incident Response Playbook Overview for Automated Quarantining:

Purpose: The purpose of this playbook is to provide a simple response sequence when a host is considered compromised and needs immediate network isolation. The workflow focuses specifically on **containment and asset isolation.**

Trigger: The process begins when a critical compromise indicator identifies a host that requires containment: Critical Compromise Indicator -> Identify Affected Host -> Trigger Containment.

Revoke Active Sessions: The first simulated containment action is: **Revoke active sessions** . This represents removing existing authenticated sessions associated with the compromised host: Affected Host -> Active Sessions -X Revoked

Apply Quarantine Rules: The next action is to place the affected system into a restricted network state: Compromised Host -> QUARANTINE Security Group -> Restricted Network Access. The material describes automated isolation through mechanisms such as Security Groups and network controllers.

Restricted External Egress: The workflow then simulates null-routing external outbound traffic: Compromised Host --X External Network. This represents preventing the affected system from sending traffic outside its permitted network area.

Confirm Isolation: The simulated script finishes by reporting: “CONTAINMENT SUCCESSFUL“ and confirms that the host is isolated from the network segment.

Final Project: Automated web Vulnerability Scanner:

Overview:

Phase 2 focused on web application security, infrastructure assessment, defensive coding, security automation, and incident-response concepts. Across Days 16–30, Python was used to build small defensive tools covering reconnaissance, vulnerability detection, secure configuration, monitoring, reporting, alerting, and containment. The work progressed from individual security checks toward a final modular scanner that combined several of the earlier capabilities into one workflow.

Day 16 — HTTP Security Header Analysis

A Python script using the requests library was used to analyze important HTTP response headers, including Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, and X-Content-Type-Options. The exercise demonstrated how CSP can restrict untrusted resource loading and provide an additional defensive layer against XSS-related activity, while also showing that missing security headers represent absent protections rather than proof of an exploitable vulnerability.

Day 17 — Local Network Port & Service Scanning

A socket-based Python scanner was used against 127.0.0.1 to identify locally exposed TCP services. A local HTTP server running on port 8080 provided a controlled test service, allowing the scanner to successfully detect an open port and demonstrate how network-service exposure can be mapped within a development environment.

Day 18 — SQL Injection Log Detection

A log-analysis engine was created with Python and regular expressions to detect suspicious SQL injection patterns in simulated web-access logs. The tool identified indicators such as logical SQL conditions, UNION SELECT, quotation characters, and comment sequences, while the analysis also demonstrated why prepared statements are safer than directly combining user input with executable SQL. Document 25

Day 19 — Docker Misconfiguration Scanner

A static Dockerfile analysis tool was developed to detect insecure configuration patterns such as the use of an unpinned latest image, exposure of SSH port 22, and the absence of an explicit USER directive. The project demonstrated how automated configuration review can identify deployment risks such as build drift, unnecessary network exposure, and implicit root execution.

Day 20 — Web Directory Discovery

Using Python, the requests library, and a local Flask server, a directory-discovery simulation tested predefined routes such as /admin, /dashboard, /.env, and /backup.sql. The project demonstrated how HTTP response differences can reveal accessible or restricted resources and highlighted the importance of explicit routing controls and protecting sensitive files from public access.

Day 21 — XSS Payload Sanitization

An XSS sanitization function was implemented using Python's html and re modules. The function encoded HTML structural characters and replaced selected XSS-related tokens, demonstrating how potentially unsafe input can be neutralized before reaching the browser while also introducing the differences between Stored, Reflected, and DOM-based XSS.

Day 22 — API Rate Limiting

A Token Bucket rate limiter was created to control rapid client requests using a fixed token capacity and timed replenishment. The test demonstrated that initial requests were allowed until the available tokens were exhausted, subsequent rapid requests were denied, and access became possible again after tokens were replenished. This illustrated how rate limiting can help protect application services from excessive automated traffic.

Day 23 — PostgreSQL Credential Auditing

A simulated PostgreSQL credential-auditing routine was developed to identify dangerous default credentials such as postgres:postgres. The project also examined broader database-security controls including least privilege, restricted connections, separation of administrative and application accounts, and protection of sensitive database information.

Day 24 — Threat Intelligence Automation

A simulated threat-intelligence pipeline processed IP indicators and assigned actions according to their risk scores. High-risk indicators were classified as blocking candidates while lower-risk entries were logged for monitoring, demonstrating the basic automation flow of ingesting intelligence, evaluating risk, making a defensive decision, and recording the result.

Day 25 — File Upload Magic-Byte Validation

A Python validator inspected the binary signatures of PNG and JPEG files rather than trusting filename extensions alone. Testing with genuine images and a fake .jpg file demonstrated that magic-byte inspection can distinguish actual file content from misleading extensions and provides an additional security control for file-upload validation.

Day 26 — Custom Web Application Firewall

A lightweight WAF engine was developed using Python and regular expressions to inspect simulated HTTP requests for XSS, SQL injection, and path-traversal patterns. The tool demonstrated basic allow-or-block decision logic and introduced both inline and reverse-proxy WAF architectures as methods for inspecting traffic before it reaches backend application processing.

Day 27 — Vulnerability Report Aggregation

A Python and JSON-based vulnerability aggregator demonstrated how findings from different security tools can be collected into a unified structure. Using a simulated high-severity hardcoded-password finding, the project showed how centralized reporting can reduce fragmented results and make security findings easier to review, prioritize, and communicate with technical and non-technical stakeholders.

Day 28 — SIEM Webhook Alert Automation

A simulated SIEM alert workflow converted a critical security event into a structured JSON webhook payload containing the event type and source IP address. The workflow demonstrated how alerts could move from a detection system to collaboration

platforms such as Slack, Microsoft Teams, or PagerDuty, while the actual HTTP POST remained disabled to keep the task controlled.

Day 29 — Incident Containment & Asset Isolation

An automated containment workflow was simulated for a compromised host. The script represented revoking active sessions, applying quarantine Security Group rules, restricting external egress traffic, and confirming host isolation, demonstrating how automated containment can help reduce lateral movement during an incident. The implementation remained a simulation and did not modify real firewall, routing, cloud, or session configurations.

Day 30 — Automated Web Vulnerability Scanner

The final project combined several earlier techniques into a single modular Python application. The reconnaissance module performs selected local port scanning and directory discovery, while the vulnerability-audit module demonstrates SQL injection log detection and XSS sanitization. Findings are collected in a common structure and exported to a JSON security report.

The final scanner demonstrates how separate defensive scripts can be integrated into a larger security workflow:

Local Target -> Reconnaissance -> Vulnerability Auditing -> Findings Collection -> JSON Security Report

This final project brought together reconnaissance, defensive analysis, structured reporting, and modular software design while remaining within a controlled local testing environment.

Final Conclusion

Phase 2 developed practical experience across web security, network reconnaissance, secure coding, container and database security, threat intelligence, defensive filtering, vulnerability reporting, SIEM concepts, and incident response. The main progression was from building individual defensive scripts to understanding how these capabilities can work together as part of an automated security workflow.

The final scanner represents that progression by integrating selected reconnaissance, vulnerability-analysis, and reporting capabilities into one application. Together, the Phase 2 projects demonstrate both the technical implementation of defensive security controls and an understanding of how those controls contribute to broader security monitoring, prevention, and response.